



UNIVERSIDADE ESTADUAL DE SANTA CRUZ

FLÁVIA ALESSANDRA SANTOS DE JESUS

**DESENVOLVIMENTO DE UM SISTEMA WEB MONTADOR
DE BAREMA DE ATIVIDADES COMPLEMENTARES PARA
O CURSO DE CIÊNCIA DA COMPUTAÇÃO DA UESC**

ILHÉUS - BAHIA

2026

FLÁVIA ALESSANDRA SANTOS DE JESUS

**DESENVOLVIMENTO DE UM SISTEMA WEB MONTADOR
DE BAREMA DE ATIVIDADES COMPLEMENTARES PARA
O CURSO DE CIÊNCIA DA COMPUTAÇÃO DA UESC**

Trabalho de Conclusão de Curso submetido à Universidade Estadual de Santa Cruz, como requisito para obtenção do grau de Bacharel em CIÊNCIA DA COMPUTAÇÃO.

Orientador: Álvaro Vinícius de Souza Coelho.

**ILHÉUS - BAHIA
2026**

FLÁVIA ALESSANDRA SANTOS DE JESUS

**DESENVOLVIMENTO DE UM SISTEMA WEB MONTADOR
DE BAREMA DE ATIVIDADES COMPLEMENTARES PARA
O CURSO DE CIÊNCIA DA COMPUTAÇÃO DA UESC**

**Trabalho de Conclusão de Curso subme-
tido à Universidade Estadual de Santa
Cruz como requisito para obtenção do
grau de Bacharel em CIÊNCIA DA
COMPUTAÇÃO.**

Ilhéus, Julho de 2026.

Orientador(a): Álvaro Vinícius de Souza
Coelho
Universidade Estadual de Santa Cruz

Prof^a. Martha Ximena Torres Delgado
Universidade Estadual de Santa Cruz

Prof.
Universidade Estadual de Santa Cruz

Agradecimentos

Resumo

Palavras-chave: .

Abstract

Keywords: .

Lista de ilustrações

Figura 1 – Fluxo de desenvolvimento adotado na construção do protótipo.	28
Figura 2 – Diagrama de Casos de Uso do sistema Barema.	32
Figura 3 – Diagrama de Classes do núcleo de domínio do sistema.	35
Figura 4 – Modelo Entidade-Relacionamento (MER) do banco de dados.	36
Figura 5 – Tela inicial do sistema Barema.	37
Figura 6 – Formulário do barema com atividades carregadas.	37
Figura 7 – Tela de <i>login</i> administrativo.	39
Figura 8 – Painel do coordenador para análise das solicitações.	40

Lista de abreviaturas e siglas

Sumário

1	INTRODUÇÃO	13
1.1	Objetivos	13
1.1.1	Objetivo Geral	14
1.1.2	Objetivos Específicos	14
1.2	Contribuições do Trabalho	14
1.3	Organização do Documento	15
2	PROBLEMA	16
2.1	Definição	16
2.2	Desafios	17
2.2.1	Regras de Negócio	18
2.2.2	Tempo de Resposta	18
2.2.3	Manipulação de Arquivos	19
2.2.4	Extração e Pré-validação Automatizada de Dados	20
3	JUSTIFICATIVA	22
3.1	Âmbito Administrativo	22
3.2	Meio Acadêmico	22
3.3	Justificativa Técnica	23
4	FUNDAMENTAÇÃO TEÓRICA	24
4.1	Arquitetura em Camadas e o Padrão MVC	24
4.2	Mapeamento Objeto-Relacional e Persistência de Dados	24
4.3	Processamento de Documentos Digitais e Extração de Texto	25
4.4	Reatividade de Interface e Tempo de Resposta	25
5	TRABALHOS CORRELATOS	26
6	METODOLOGIA	27
6.1	Caracterização do Trabalho	27
6.2	Procedimentos Metodológicos	27
6.2.1	Prototipagem Evolutiva	27
6.2.2	Levantamento de Requisitos	28
6.2.3	Definição da Arquitetura, Ferramentas e Implementação	29
6.3	Estratégia de Avaliação	29
6.3.1	Critérios de Sucesso do Protótipo	30

7	SOLUÇÃO PROPOSTA	32
7.1	Visão Geral e Casos de Uso	32
7.2	Requisitos do Sistema	33
7.3	Arquitetura e Persistência de Dados no Projeto	34
7.4	Implementação	36
7.4.1	Montagem do Barema	36
7.4.2	Extração de Dados	38
7.4.3	Pré-validação e Geração do Documento	38
7.4.4	Painel de Análise (Coordenação)	39
8	RESULTADOS E DISCUSSÃO	41
8.1	Resultados Obtidos	41
8.2	Validação do Protótipo	42
8.3	Discussão da Solução	42
8.4	Limitações Observadas	43
9	CONCLUSÃO	44
	REFERÊNCIAS	46

1 Introdução

O presente trabalho aborda a concepção e a construção de um sistema *web* de apoio à preparação do Barema de Atividades Complementares do curso de Ciência da Computação da Universidade Estadual de Santa Cruz (UESC). As atividades complementares configuram um requisito obrigatório para a integralização curricular do curso. O reconhecimento dessas horas, contudo, depende da apresentação de um documento unificado que reúna os comprovantes, calcule a carga horária e valide as categorias estritamente segundo o regulamento do Colegiado do Curso (COLCIC).

Atualmente, o processo de elaboração desse Barema é eminentemente manual e impõe uma carga cognitiva e operacional significativa ao discente, que precisa:

- reunir certificados e comprovantes dispersos em formato PDF;
- ordenar e organizar estruturalmente esses documentos;
- calcular individualmente as cargas horárias de cada certificado;
- aplicar os limites e tetos máximos de horas previstos por categoria;
- preencher uma tabela de indexação de páginas de forma manual para guiar a auditoria.

Esse fluxo braçal gera três ramificações problemáticas principais. Primeiro, eleva a suscetibilidade a erros aritméticos e de extrapolação dos tetos e pisos regulamentares, o que frequentemente invalida a submissão. Segundo, engessa a edição do documento, visto que pequenas alterações na ordem ou no conjunto de arquivos exigem a revisão completa da paginação e do índice. Terceiro, sobrecarrega o escopo administrativo da coordenação, que se vê obrigada a devolver e reavaliar inúmeros processos devido a falhas estritamente formais antes de poder analisar o mérito acadêmico.

Diante desse cenário, a solução proposta não busca substituir a análise de mérito soberana do colegiado, mas atuar como uma camada tecnológica de contingência e apoio. O propósito é oferecer uma ferramenta capaz de mitigar erros formais primários, aumentando a confiabilidade do documento submetido e otimizando o fluxo de trabalho institucional.

1.1 Objetivos

Para guiar o desenvolvimento da solução proposta e garantir que ela atenda às necessidades do domínio acadêmico, o trabalho foi estruturado em torno dos seguintes

objetivos.

1.1.1 Objetivo Geral

Desenvolver um protótipo funcional de sistema *web* para a automação, pré-validação e geração padronizada de Baremas de Atividades Complementares voltado aos discentes e à coordenação do COLCIC/UESC.

1.1.2 Objetivos Específicos

Para alcançar o objetivo central, delinearam-se as seguintes metas específicas:

1. Realizar o levantamento de requisitos funcionais, normativos e operacionais em conjunto com a coordenação do COLCIC e egressos do curso.
2. Desenvolver a aplicação *web* com capacidade de orquestrar a extração de dados, aplicar regras de negócio e compilar um arquivo PDF unificado, indexado e paginado.
3. Implementar uma arquitetura de banco de dados para a persistência das solicitações, garantindo a rastreabilidade do processo e a automação do envio de *feedbacks*.
4. Validar a precisão computacional do protótipo, atestando a conformidade dos cálculos de horas, a detecção de duplicidades e a aplicação dos tetos e pisos regulamentares do curso.

1.2 Contribuições do Trabalho

Como contribuição para a comunidade acadêmica e para a área de Engenharia de Software aplicada à gestão educacional, este trabalho apresenta:

- um modelo de orquestração arquitetural otimizado para a manipulação e extração de metadados de arquivos PDF em estado transiente (na memória);
- um mecanismo algorítmico de pré-validação que cruza as informações declaradas pelo usuário com os dados brutos extraídos (inclusive via OCR) dos certificados;
- a modelagem de um motor de regras de negócio escalável, capaz de respeitar tetos numéricos e categorias normativas;
- um ecossistema funcional com interface reativa para o discente e um painel de gerenciamento administrativo com gatilhos de comunicação integrados para a coordenação.

1.3 Organização do Documento

Este documento está estruturado em capítulos organizados da seguinte forma. O Capítulo 2 apresenta o detalhamento do problema e os desafios do processo atual. O Capítulo 3 expõe a justificativa administrativa, acadêmica e técnica que motiva o projeto. O Capítulo 4 apresenta a fundamentação teórica, abordando os conceitos e as tecnologias que embasam o desenvolvimento da ferramenta. O Capítulo 5 discute os trabalhos correlatos, comparando a abordagem deste projeto com outras soluções existentes. O Capítulo 6 descreve a metodologia, o levantamento de requisitos e as estratégias adotadas para a construção do protótipo. O Capítulo 7 detalha a arquitetura e a engenharia da solução técnica implementada. O Capítulo 8 apresenta a avaliação dos cenários de teste, discutindo a aplicabilidade e o atendimento aos critérios de sucesso. O capítulo 9 sintetiza as conclusões obtidas e indicam diretrizes para trabalhos futuros. E por fim, as referências bibliográficas complementam o documento, fornecendo o suporte teórico para o desenvolvimento.

2 Problema

Este capítulo aborda a conceituação do problema para a solução proposta, bem como seus desafios.

2.1 Definição

As Atividades Complementares (AC) constituem um requisito obrigatório para a integralização curricular no curso de Ciência da Computação da UESC. Seu cumprimento envolve a participação do discente em eventos científicos, estágios, projetos de extensão, monitorias e outras vivências formativas. Para que essas horas sejam reconhecidas oficialmente pelo Colegiado do Curso (COLCIC), o aluno deve organizar e submeter o Barema: um documento unificado que reúne os comprovantes das atividades realizadas, categoriza-os segundo as normas vigentes e totaliza a carga horária acumulada por categoria.

O processo de elaboração desse documento, contudo, é inteiramente manual e descentralizado. Não existe ferramenta oficial fornecida pela instituição para auxiliar nessa tarefa. O discente precisa reunir individualmente cada certificado em formato PDF, organizá-los em um único arquivo, preencher manualmente uma tabela de indexação — indicando em qual página do documento unificado cada comprovante se encontra — e calcular as cargas horárias respeitando os tetos máximos permitidos por categoria, conforme as regras específicas do curso de bacharelado em Ciência da Computação da UESC. Cada uma dessas etapas é executada de forma independente, sem qualquer validação automatizada, e qualquer alteração na ordem ou no conjunto de documentos pode gerar um efeito cascata de erros que inviabiliza o aproveitamento da indexação previamente realizada.

Esse cenário resulta em três problemas concretos e recorrentes. O primeiro é a complexidade regulatória: os alunos encontram dificuldade em aplicar corretamente as regras de carga horária definidas pelo COLCIC — como o limite de horas permitido por categoria de atividade —, o que leva a cálculos errôneos e à inclusão indevida de horas que excedem os tetos regulamentares. O segundo é a propensão a erros de indexação: o preenchimento manual da coluna de páginas é suscetível a falhas simples, porém de alto impacto — um único certificado inserido ou removido do conjunto invalida todos os números de página subsequentes, obrigando o discente a reiniciar o processo de edição, reordenação e reescrita da tabela integralmente. Relatos colhidos junto a discentes evidenciam casos em que o documento precisou ser refeito três vezes consecutivas por conta exclusivamente desse tipo de inconsistência. O terceiro problema é o retrabalho administrativo gerado na coordenação: o envio de baremas incorretos — seja por erros de cálculo, por indexação

equivocada ou por separação indevida entre o barema e os documentos comprobatórios — obriga a coordenação a realizar múltiplas rodadas de revisão com cada aluno, atrasando o aproveitamento das atividades e, conseqüentemente, o processo de colação de grau.

É importante destacar que as soluções genéricas de manipulação de PDF disponíveis no mercado, como o (iLovePDF 2026), não resolvem esse problema. De acordo com sua documentação oficial, essa ferramenta opera sobre a estrutura do arquivo, permitindo unir, dividir ou comprimir documentos. Embora atualmente ela ofereça recursos para resumir ou comparar textos, sua lógica é de propósito geral, desconhecendo as regras específicas para a montagem do Barema de AC, ou outro contexto específico. Essa limitação evidencia que tais ferramentas funcionam apenas como camadas tecnológicas isoladas, falhando em atuar como um Sistema de Informação eficaz que, segundo (Laudon e Laudon 2014), deve compreender a organização e as regras que o cercam para atingir seus objetivos. No caso do COLCIC, essas ferramentas permitem unir arquivos PDF, mas não calculam cargas horárias, não aplicam tetos regulamentares e não geram a tabela de indexação automaticamente. A responsabilidade sobre toda a lógica de negócio permanece, portanto, integralmente com o aluno. O problema não é a ausência de uma ferramenta de junção de arquivos — é a ausência de um sistema que compreenda as regras do processo e as aplique de forma integrada à manipulação dos documentos.

2.2 Desafios

O desenvolvimento do sistema de apoio ao Barema do COLCIC é fundamentado na criação de uma solução que integra a manipulação de documentos digitais com a aplicação rigorosa de normas acadêmicas. Para que a ferramenta cumpra sua função, ela deve operar como um ambiente de processamento inteligente, capaz de coordenar dados binários e regras de negócio sem a necessidade de uma infraestrutura de autenticação persistente para o discente. Esta arquitetura é sustentada por um orquestrador de processos, cuja missão é gerenciar fluxos simultâneos de dados, garantindo que arquivos desestruturados sejam convertidos em um documento técnico validado e em conformidade com a regulamentação do curso.

A complexidade deste orquestrador reside na necessidade de manter a integridade das informações em um cenário de processamento transiente. Diferente de sistemas convencionais que armazenam cada etapa no banco de dados, esta solução utiliza uma estrutura de dados volátil na memória do servidor para gerenciar o que se define como estado do arquivo. A robustez do sistema é demonstrada em sua capacidade de manter esses cálculos íntegros mesmo diante de variações ou erros na entrada de dados pelo usuário.

Dessa forma, o sistema se diferencia de um concatenador comum de arquivos por realizar uma fusão assistida. Enquanto ferramentas simples apenas unem documentos, o

orquestrador proposto executa uma pré-validação automatizada, confrontando as declarações do estudante com os dados detectados nos certificados e aplicando restrições de carga horária em tempo de resposta imediato. Este fluxo lógico, que assegura a confiabilidade regulatória do produto final, é detalhado a seguir por meio dos quatro desafios principais enfrentados na implementação da ferramenta.

2.2.1 Regras de Negócio

Um dos desafios é a codificação das regras de negócio do COLCIC em lógica computacional, especificamente no que tange aos pisos e tetos de carga horária. Um teto de carga horária é definido como o limite máximo de horas que uma determinada categoria de atividade (como pesquisa, extensão ou monitoria) pode integralizar no currículo do discente. Mesmo que o aluno possua certificados que somem, por exemplo, 200 horas em eventos, se o regulamento estabelece um teto de 60 horas para essa categoria, o sistema deve ser capaz de reconhecer esse limite e considerar apenas o valor permitido para o cálculo final, evitando que o discente ultrapasse as metas estabelecidas pelo curso. De maneira contrária, o piso refere-se ao limite mínimo de horas que uma determinada categoria de atividade exige para ser computada.

Essas normas são representadas no sistema não apenas como validações simples, mas como um dicionário de regras estruturado dentro da lógica do orquestrador. Para garantir a transparência dessa implementação, as regras de negócio e o fluxo de validação estão detalhadamente documentados através de diagramas lógicos. O mapeamento completo das categorias e seus respectivos limites pode ser visualizado no Diagrama de Regras de Negócio, localizado na Seção ?? (página ??), que ilustra como o sistema intercepta os dados extraídos e aplica as restrições regulamentares de forma automatizada.

A importância dessa precisão é crítica, pois qualquer imprecisão na tradução do regulamento para o código compromete a confiabilidade de todo o processo. Como o Barema gerado possui valor regulatório e serve de base para o deferimento da colação de grau pelo Colegiado, a lógica computacional deve refletir fielmente o edital. Erros de somatório ou falhas na aplicação dos tetos e pisos poderiam gerar documentos inconsistentes, resultando em retrabalho administrativo e prejuízos acadêmicos para o discente, o que reforça a necessidade de uma codificação rigorosa e testada.

2.2.2 Tempo de Resposta

O sistema baseia-se em um modelo de computação reativa, onde a coordenação de operações deve ocorrer em tempo de resposta crítico. Diferente de sistemas de tempo real crítico, onde o atraso implica em falha catastrófica (Tanenbaum e Bos 2016), este projeto foca na manutenção da fluidez da experiência do usuário, buscando respeitar os limites

de interatividade estabelecidos por Nielsen (1993), onde respostas em até 0,1 segundo sustentam a percepção de instantaneidade, até 1,0 segundo mantêm o fluxo de pensamento do usuário sem interrupções e 10 segundos é o limite para manter a atenção do usuário focada no diálogo (Nielsen 1993). Essa necessidade manifesta-se no processamento imediato de arquivos: ao anexar um documento, o backend deve extrair metadados e recalcular as regras de teto de horas instantaneamente, assegurando que o estado visual da interface reflita fielmente o estado lógico da aplicação.

É necessário distinguir as operações de interface das operações de processamento profundo. Enquanto validações de regras de negócio simples — como o respeito aos limites mínimos e máximos de carga horária — devem ocorrer no limite da percepção de instantaneidade (0,1s), a pré-validação automatizada de certificados (envolvendo OCR e IA) demanda um tempo de resposta superior. O desafio técnico reside em garantir que essa latência estendida não resulte em incerteza para o discente, utilizando padrões de comunicação assíncrona para fornecer estados intermediários de processamento enquanto a extração é concluída no backend.

Para mitigar o impacto da latência no processamento de arquivos binários, o sistema utiliza padrões de comunicação assíncrona. Operações que demandam processamento intensivo de visão computacional e IA - como a pré-validação de certificados - são desacopladas da requisição principal, permitindo que o usuário continue a montagem do Barema enquanto as tarefas são executadas em segundo plano. O uso de técnicas como Polling assegura que a interface reativa receba atualizações assim que a extração de metadados é concluída, respeitando as diretrizes de feedback visual para processos de longa duração.

2.2.3 Manipulação de Arquivos

O primeiro desafio técnico reside na manipulação de arquivos PDF em ambiente web sem a necessidade de contas de usuário. Como o sistema não exige login, ele opera através de um fluxo de processamento temporário. O estado do arquivo - estrutura de dados temporária que reside na memória da aplicação durante a sessão de uso - não é salvo permanentemente no banco de dados assim que o aluno faz o upload; em vez disso, ele existe apenas na memória da aplicação enquanto o aluno organiza seu Barema. Nesse estágio, o sistema captura metadados fundamentais:

1. Categoria e Ordem: O sistema monitora a interface em tempo real através de eventos de JavaScript. Quando o aluno insere um arquivo para uma categoria - atividade do barema - ou altera sua posição, o navegador captura o ID da categoria e o índice da lista, enviando esses dados ao servidor via requisições assíncronas (Fetch API). No backend, o orquestrador organiza essas informações em uma estrutura de lista na memória temporária para manter a hierarquia definida pelo usuário.

2. Número de Páginas: No momento do upload, antes mesmo de salvar o arquivo permanentemente, o servidor utiliza a biblioteca PyMuPDF para abrir o fluxo de bytes do PDF diretamente da memória. O sistema acessa o cabeçalho do documento para extrair a propriedade de contagem de páginas de forma instantânea, permitindo que o contador visual da interface seja atualizado sem atrasos percebidos.
3. Cálculo de Tetos: Com os metadados de categoria e carga horária carregados na memória, o orquestrador executa uma função de soma para cada grupo de certificados. O sistema compara esses totais com um dicionário de regras pré-definidas (limites do edital) e devolve uma resposta imediata ao frontend. Caso um limite seja atingido, o JavaScript altera o estado visual da página, exibindo alertas ou mudando cores de destaque para orientar o aluno.

É importante destacar que o banco de dados (tabela análises) só é acionado se o usuário decidir converter esse rascunho em uma solicitação formal de análise. Somente após essa escolha o sistema unifica os arquivos em um documento final, gera o link de acesso e persiste as informações de matrícula e status no SQLite. Essa abordagem garante que o servidor não fique sobrecarregado com arquivos de tentativas incompletas, respeitando a natureza imediata e utilitária da ferramenta.

2.2.4 Extração e Pré-validação Automatizada de Dados

O terceiro desafio reside na extração e pré-validação automatizada de dados, um processo que visa transformar documentos PDF brutos em informações estruturadas. Ao receber um certificado, o sistema realiza uma análise preliminar para verificar a autenticidade e extrair dados críticos, como a carga horária e a data de emissão.

Essa tarefa introduz a complexidade de lidar com documentos não estruturados, uma vez que cada instituição emissora de certificados utiliza layouts e linguagens distintas. Para superar essa barreira, o projeto utiliza técnicas de processamento de texto e expressões regulares (Regex). O sistema varre o conteúdo textual extraído pelo PyMuPDF em busca de padrões numéricos associados a palavras-chave (como "horas", "h" ou "carga horária"), permitindo que a aplicação compreenda o conteúdo do certificado sem a necessidade de intervenção humana imediata.

Além da extração, a pré-validação atua na mitigação de erros antes da submissão final ao COLCIC. O sistema confronta automaticamente o que o discente declara no formulário com o que foi detectado no arquivo anexo. Caso haja uma divergência — por exemplo, o aluno declarar 40 horas em um certificado que o sistema identificou como tendo apenas 10 horas —, alertas de irregularidade são inseridos no documento final. Essa abordagem garante que o Barema unificado chegue ao coordenador com um nível de

precisão muito superior, reduzindo o retrabalho e as inconsistências no processo de análise acadêmica.

3 Justificativa

A transição para um modelo computacional especializado não visa apenas a digitalização de documentos, mas a garantia da integridade dos dados e da otimização de processos por meio da Engenharia de Software, que consiste na aplicação de uma abordagem sistemática e disciplinada para o desenvolvimento de soluções confiáveis (Pressman e Maxim 2016). A necessidade deste sistema fundamenta-se na aplicação de soluções automatizadas como resposta a processos manuais ineficientes, estruturando-se em duas frentes: a otimização administrativa de recursos e a mitigação de riscos acadêmicos para o estudante.

3.1 Âmbito Administrativo

Do ponto de vista administrativo, a coordenação do curso opera com recursos humanos limitados e precisa analisar individualmente cada barema submetido. O volume de documentos incorretos que retornam para correção representa um custo operacional real: cada revisão exige nova comunicação com o discente, novo prazo de reenvio e nova análise pelo coordenador.

Nesse cenário, a automação do fluxo não é apenas uma conveniência, mas uma aplicação direta de princípios consolidados na Engenharia de Software. A automação minimiza a intervenção humana em tarefas repetitivas, reduzindo o risco de erros operacionais e aumentando a eficiência (Abidemi 2024). Assim, a indexação de páginas e o cálculo de cargas horárias são tarefas determinísticas e com regras bem definidas, beneficiando-se diretamente dessa abordagem para otimizar a capacidade de atendimento do colegiado.

3.2 Meio Acadêmico

Do ponto de vista acadêmico, o processo atual impõe ao discente um ônus desproporcional à natureza da tarefa. Montar um barema não é uma atividade de aprendizado — é uma tarefa burocrática de baixa complexidade cognitiva, mas de alto potencial de erro quando executada manualmente. Exigir que o aluno dedique tempo e atenção a contar páginas e somar horas em planilhas é desviar energia que poderia ser investida nas próprias atividades complementares que o processo visa reconhecer. Além disso, erros nessa etapa têm consequências concretas: atrasos no aproveitamento das AC podem comprometer o cumprimento dos prazos para a colação de grau, impactando diretamente o planejamento acadêmico e profissional do estudante.

3.3 Justificativa Técnica

Por fim, a justificativa técnica do projeto reside no fato de que a solução proposta não é apenas um agregador de arquivos, mas um sistema que codifica as regras de negócio do COLCIC. Isso significa que as validações de carga horária máxima por categoria, a geração automática da tabela de indexação e a unificação dos comprovantes ocorrem de forma integrada e transparente para o usuário. O resultado é um documento gerado com garantia de conformidade regulatória — algo que nenhuma ferramenta genérica hoje disponível é capaz de oferecer para esse contexto específico.

4 Fundamentação Teórica

Este capítulo apresenta a base conceitual que sustenta o desenvolvimento e a arquitetura do sistema de apoio ao Barema do COLCIC. São abordados os princípios de arquitetura de sistemas para a Web, os conceitos de processamento de documentos binários e mineração de texto, além dos fundamentos de *interfaces* reativas e persistência de dados.

4.1 Arquitetura em Camadas e o Padrão MVC

O desenvolvimento de aplicações web modernas exige a adoção de modelos arquiteturais que favoreçam a modularidade e o isolamento de responsabilidades. Tradicionalmente, o padrão arquitetural em camadas organiza o sistema em níveis distintos — apresentação, negócio e persistência —, de forma que alterações em uma camada não comprometam as demais. Dentro desse modelo, o padrão *Model-View-Controller* (MVC) é amplamente empregado para separar a lógica de apresentação da lógica de negócio e de acesso a dados (Pressman e Maxim 2016).

Fowler descreve o MVC como um dos padrões arquiteturais mais relevantes para aplicações web, destacando que a separação entre o modelo de dados, a camada de visualização e o controlador de fluxo é o que permite a evolução independente de cada componente sem ruptura do sistema como um todo (?). No contexto deste trabalho, essa separação é crítica: as regras de negócio do COLCIC precisam estar isoladas da interface e da camada de geração de documentos, de modo que alterações no regulamento possam ser incorporadas sem reescrever o sistema inteiro.

4.2 Mapeamento Objeto-Relacional e Persistência de Dados

A camada de persistência de dados em sistemas orientados a objetos frequentemente enfrenta o problema da incompatibilidade de impedância, caracterizado pela diferença entre o modelo conceitual de objetos na memória e o modelo relacional das tabelas em um banco de dados SQL. Para mitigar esse problema, utiliza-se o Mapeamento Objeto-Relacional (*Object-Relational Mapping* — ORM).

O ORM atua como uma camada de abstração que traduz de forma transparente as operações sobre objetos computacionais em comandos de manipulação de dados em bancos relacionais (?). Essa abstração permite que a aplicação opere sobre estruturas nativas da linguagem — como dicionários e listas de objetos em Python — e delegue ao ORM a

responsabilidade de sincronizar essas estruturas com o banco de dados, minimizando o acoplamento direto com o motor de persistência escolhido.

4.3 Processamento de Documentos Digitais e Extração de Texto

A manipulação de arquivos no formato PDF (*Portable Document Format*) introduz desafios relacionados à natureza não estruturada das informações. Diferente de linguagens de marcação estruturadas (como XML ou JSON), o PDF foca na fidelidade de renderização visual e na disposição física de caracteres em coordenadas bidimensionais, omitindo frequentemente tags de estrutura semântica ou separadores lógicos de parágrafos.

Para transformar esses fluxos de bytes brutos em dados inteligíveis pela lógica de negócio, utilizam-se bibliotecas especializadas na decodificação da estrutura interna do arquivo. A partir do conteúdo textual extraído, aplicam-se técnicas de processamento baseadas em Expressões Regulares (*Regular Expressions* — Regex). As expressões regulares operam como autômatos finitos determinísticos que realizam o casamento de padrões textuais (HOPCROFT, MOTWANI e ULLMAN 2006), permitindo varrer o conteúdo do documento em busca de tokens numéricos adjacentes a qualificadores de carga horária — como "horas", "h" e "carga horária" — para identificar automaticamente os metadados operacionais dos certificados

4.4 Reatividade de Interface e Tempo de Resposta

A experiência do usuário em plataformas utilitárias é diretamente influenciada pela latência observada nas atualizações da interface. Nielsen estabelece três limiares fundamentais de tempo de resposta que permanecem relevantes desde suas formulações originais: respostas em até 0,1 segundo sustentam a percepção de instantaneidade; respostas em até 1,0 segundo mantêm o fluxo de pensamento do usuário sem interrupções perceptíveis; e respostas que ultrapassam 10 segundos comprometem a atenção do usuário e exigem *feedback* visual explícito de progresso (Nielsen 1993).

Para alcançar esse patamar de reatividade em operações de manipulação de arquivos, adota-se o modelo de computação assíncrona na camada de apresentação. Utilizando *APIs* nativas dos navegadores modernos, como a *Fetch API*, a interface web envia dados e arquivos binários em segundo plano para as rotas do servidor. O servidor processa as informações e devolve respostas leves em formato JSON, permitindo que o estado visual do navegador seja modificado dinamicamente via JavaScript sem a necessidade de recarregamento completo da página e sem interromper o fluxo de pensamento do usuário.

5 Trabalhos Correlatos

6 Metodologia

Este capítulo descreve o caminho adotado para conduzir o trabalho e para avaliar se a solução proposta atende ao problema identificado. Para isso, o texto apresenta inicialmente a natureza do projeto necessária para compreensão do problema, seguida pelos procedimentos de desenvolvimento e, por fim, a estratégia de avaliação do protótipo.

6.1 Caracterização do Trabalho

O trabalho caracteriza-se como uma atividade de natureza tecnológica, pois tem como objetivo produzir uma solução computacional para apoiar um processo acadêmico real: a organização e a pré-validação do barema de atividades complementares do Colegiado de Ciência da Computação. O projeto possui abordagem qualitativa, uma vez que parte da análise do fluxo utilizado pelos discentes, das regras institucionais e das dificuldades observadas na preparação dos documentos.

O foco do estudo não é propor um novo método teórico de avaliação acadêmica, mas transformar regras já existentes em uma ferramenta funcional. Assim, o mérito do trabalho está na construção, organização e validação de um protótipo capaz de reduzir erros recorrentes no preenchimento, anexação e conferência inicial dos certificados.

6.2 Procedimentos Metodológicos

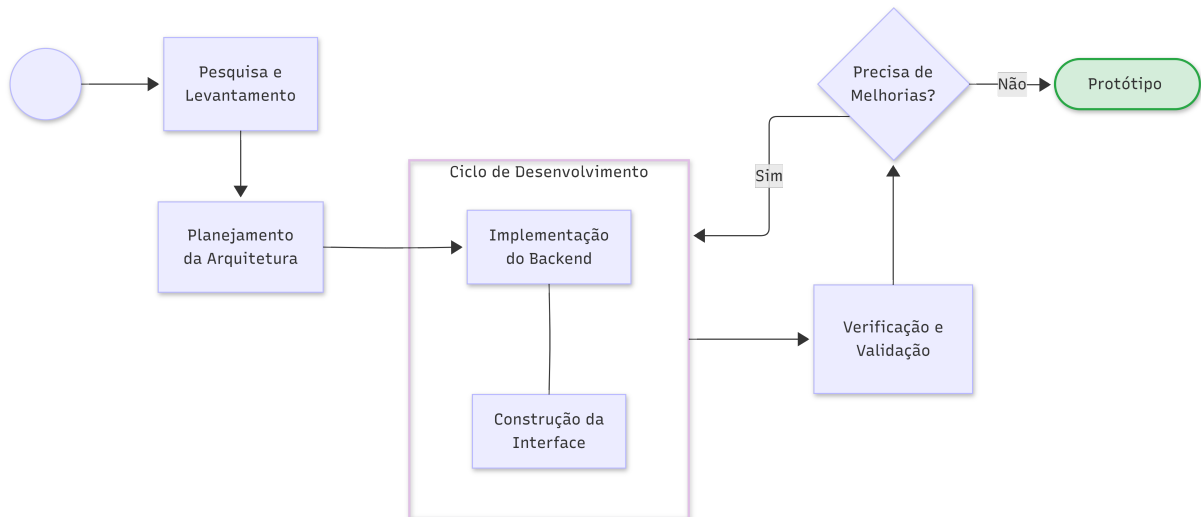
Esta seção detalha as etapas operacionais e metodológicas adotadas para a concepção e construção do sistema. Para estruturar o processo de desenvolvimento de forma lógica e rastreável, o texto apresenta inicialmente o modelo de ciclo de vida escolhido para guiar a engenharia do *software*. Na sequência, descrevem-se os passos práticos realizados, que englobam desde o levantamento técnico das regras institucionais até as diretrizes de arquitetura e codificação da ferramenta.

6.2.1 Prototipagem Evolutiva

O desenvolvimento foi conduzido por prototipagem evolutiva, em que versões sucessivas do sistema foram refinadas a partir das necessidades identificadas durante o levantamento e a implementação (Soares e Resende 2007). Essa estratégia foi adotada porque o problema envolve interação com usuário, regras de negócio específicas e tratamento de documentos em formato PDF, aspectos que exigem ajustes progressivos ao longo do desenvolvimento.

A Figura 1 sintetiza o fluxo adotado durante a construção do protótipo, evidenciando o caráter iterativo do processo. A partir do levantamento inicial de regras para o sistema, foram planejadas e implementadas versões sucessivas da ferramenta, submetidas a verificações e ajustes até que o sistema atendesse às funcionalidades previstas.

Figura 1 – Fluxo de desenvolvimento adotado na construção do protótipo.



Fonte: elaboração própria.

6.2.2 Levantamento de Requisitos

Inicialmente, foi realizado o levantamento das regras do processo de elaboração e envio do barema para elencar as funcionalidades necessárias para o sistema, compreendendo os critérios institucionais que a ferramenta precisaria respeitar. Para isso, foram conduzidas consultas junto à coordenação do COLCIC, com o objetivo de alinhar o fluxo da aplicação aos processos internos do colegiado e compreender como o sistema poderia otimizar o trabalho da coordenação. Além disso, egressos do curso foram consultados a fim de identificar pontos de atenção, dificuldades frequentes e recursos essenciais que não poderiam faltar na plataforma. A partir desse levantamento, definiram-se as principais funcionalidades do sistema, que foram formalizadas com requisitos funcionais e não funcionais na seção 7.2.

Para formalizar o entendimento dessas regras de negócio e preparar o design estrutural do sistema, adotou-se a Linguagem de Modelagem Unificada (UML). A modelagem atuou como uma ferramenta metodológica transitória: os requisitos levantados foram mapeados visualmente em Diagramas de Casos de Uso, enquanto o comportamento do estado transiente foi projetado através de Diagramas de Sequência, servindo de base documentada para a fase posterior de codificação.

6.2.3 Definição da Arquitetura, Ferramentas e Implementação

Após o levantamento, delimitou-se a estrutura macroscópica do sistema através de um modelo arquitetural em camadas baseado em *MVC*, com o propósito de isolar as responsabilidades de apresentação, controle, lógica de negócio e persistência de dados. Essa etapa de *design* estrutural foi fundamental para subsidiar a fase subsequente de desenvolvimento, garantindo que a implementação dos módulos operacionais seguisse um padrão previsível e de fácil manutenção.

Em consonância com a arquitetura definida, procedeu-se à seleção das ferramentas e do ambiente tecnológico mais adequados para suportar os requisitos do projeto. Como critério de escolha, priorizou-se um ecossistema capaz de lidar com processamento assíncrono e manipulação ágil de binários na memória. A partir dessa premissa, definiu-se a utilização da linguagem Python com o microframework Flask para a orquestração do *backend*, a biblioteca PyMuPDF para a leitura e extração de metadados dos certificados, e ferramentas *web* reativas para a construção da *interface*. Para a eventual gravação de dados, optou-se pelo banco de dados relacional SQLite, operado por meio do ORM SQLAlchemy.

A partir dos diagramas UML estruturados, delineamento arquitetural e seleção de ferramentas, iniciou-se a etapa de implementação propriamente dita (a codificação da aplicação). Nela, os componentes da arquitetura ganharam forma por meio do desenvolvimento dos mecanismos de preenchimento do formulário, da lógica de anexação de certificados, das rotinas de extração de dados e pré-validação dessas informações, culminando na lógica de montagem automatizada do arquivo final. Durante todo esse processo, as funcionalidades foram testadas de forma incremental. A cada novo recurso incorporado, foram verificados os impactos sobre o fluxo completo de uso, assegurando a convergência entre as decisões de código e os objetivos de automação estabelecidos para a plataforma.

6.3 Estratégia de Avaliação

A avaliação do protótipo foi planejada com base em testes funcionais e na verificação rigorosa de conformidade com as regras do Barema. O objetivo dessa etapa é garantir que o sistema processe corretamente as informações de entrada e aplique as validações institucionais de forma precisa, simulando o uso real por parte dos discentes e mitigando erros formais antes da submissão oficial ao colegiado.

O planejamento dos testes de conformidade consistiu no mapeamento das restrições do edital do COLCIC em matrizes de rastreabilidade. A partir dessas matrizes, estabeleceu-se que cada fluxo alternativo ou de erro desenhado para o orquestrador deveria ser provocado intencionalmente por meio de dados de entrada controlados, permitindo avaliar o comportamento adaptativo do *backend* tanto em condições ideais quanto em cenários de falha.

Para isso, a validação foi estruturada em torno de cenários específicos de uso, que cobrem as principais funcionalidades da ferramenta. Os cenários observados incluem:

- **Anexação de múltiplos certificados:** Análise de como a ferramenta gerencia, armazena temporariamente na memória e processa lotes de arquivos enviados simultaneamente.
- **Identificação de carga horária e metadados:** Teste da capacidade do motor de busca textual em extrair e contabilizar as horas contidas nos documentos binários.
- **Comparação de horas e aplicação de tetos:** Verificação do cálculo entre as horas declaradas pelo discente e o limite máximo permitido por categoria no regulamento.
- **Deteção de certificados duplicados:** Validação da regra de negócio que impede a submissão repetida do mesmo comprovante por meio da comparação do conteúdo bruto.
- **Mecanismo de contingência da pré-validação:** Avaliação da resiliência do sistema e ativação do comportamento seguro básico quando há indisponibilidade de serviços externos de inteligência artificial ou envio de arquivos sem texto extraível.
- **Geração do documento final:** Avaliação da legibilidade, paginação, inserção de separadores e integridade dos dados no PDF consolidado pela aplicação.

6.3.1 Critérios de Sucesso do Protótipo

Para determinar se a solução computacional atende satisfatoriamente ao problema identificado e valida a viabilidade tecnológica da proposta, foram estabelecidos critérios de sucesso objetivos:

- **Eficiência da *Interface*:** As atualizações de estado, substituições de arquivos e sinalizações de erro devem ocorrer de forma assíncrona, mantendo o tempo de resposta ágil e a fluidez da navegação sem recarregamento de página.
- **Precisão na Extração de Metadados:** O motor de busca baseado em *PyMuPDF* deve apresentar eficácia na leitura de certificados com camada de texto padrão, extraindo corretamente a paginação e identificando os valores de carga horária.
- **Conformidade com o Regulamento:** O sistema deve aplicar rigorosamente as restrições do COLCIC, bloqueando o cômputo de horas excedentes e gerando alertas reativos ao discente em caso de discrepâncias de nome, horas ou temporalidade.

- **Integridade da Unificação:** O arquivo PDF final gerado deve consolidar todos os anexos válidos em um único documento perfeitamente paginado, ordenado e com folhas de rosto divisórias, sem perda de qualidade visual dos binários originais.

Dessa forma, a avaliação busca responder se o protótipo reduz erros antes da submissão oficial, mantém aderência às regras do colegiado e produz um documento final devidamente organizado para a análise da coordenação.

7 Solução Proposta

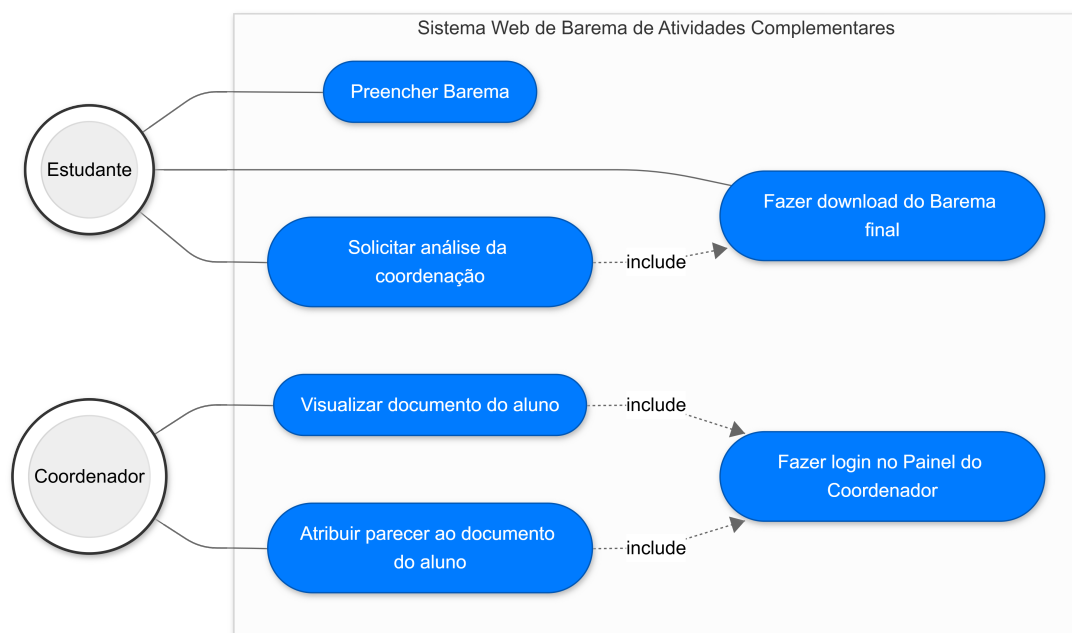
Este capítulo apresenta a solução proposta para apoiar os discentes na organização do barema de atividades complementares conforme discutido na Seção ???. A solução consiste em um sistema web que centraliza o preenchimento dos dados do aluno, a anexação dos certificados, a pré-validação das informações e a geração de um documento final em PDF.

7.1 Visão Geral e Casos de Uso

O sistema foi concebido para atuar em uma fase antecedente à submissão oficial do barema (descrita em ??), funcionando como uma camada de apoio à organização e conferência inicial dos documentos. O seu objetivo não é substituir a análise de mérito realizada pelo colegiado, mas sim atuar na mitigação ativa de erros comuns no momento do envio, como o cômputo de carga horária incompatível, a submissão de certificados repetidos e a desorganização da paginação dos anexos (melhor descritos na Seção ??).

Para mapear as interações entre os usuários e o sistema, bem como definir o escopo das funcionalidades, elaborou-se o Diagrama de Casos de Uso (Figura 2). O diagrama identifica os dois atores principais do sistema — o Discente e o Coordenador — e as ações que cada um pode realizar na plataforma.

Figura 2 – Diagrama de Casos de Uso do sistema Barema.



Fonte: Elaboração própria (2026).

A partir desta modelagem de interações, a arquitetura e a implementação do sistema foram divididas em quatro módulos operacionais lógicos: Montagem do Barema, Extração de Dados, Pré-validação, Geração do Documento e o Painel de Análise dos Baremas.

7.2 Requisitos do Sistema

Conforme delineado no capítulo 6, o levantamento de requisitos que guiou a construção dos módulos operacionais foi formalizado a seguir:

Tabela 1 – Requisitos funcionais do sistema

Código	Descrição
RF01	O sistema deve permitir que o discente anexe, categorize e altere a ordem dos certificados PDF.
RF02	O sistema deve extrair automaticamente a quantidade de páginas dos arquivos PDF enviados.
RF03	O sistema deve comparar a carga horária informada com os tetos máximos permitidos no regulamento.
RF04	O sistema deve emitir alertas visuais ao usuário caso haja divergência entre as regras e os dados inseridos.
RF05	O sistema deve consolidar os arquivos em um único PDF, com indexação e paginação automática.
RF06	O sistema deve calcular e exibir a carga horária total somada dos certificados no documento final.
RF07	O sistema deve verificar se a data de emissão ou realização do certificado está dentro do período ativo de graduação do aluno.
RF08	O sistema deve comparar se a carga horária e o nome informados no formulário coincidem com os dados extraídos do PDF.
RF09	O sistema deve detectar e alertar sobre a submissão de arquivos PDF duplicados.
RF10	O sistema deve permitir que o discente solicite a análise formal do Barema após a geração do documento.
RF11	O sistema deve fornecer um painel administrativo protegido para o coordenador visualizar e gerenciar o histórico de solicitações.
RF12	O sistema deve enviar o <i>feedback</i> da análise automaticamente para o e-mail ou WhatsApp do discente.

Fonte: Elaborado pelo autor (2026).

Também foram detectados os Requisitos Não-Funcionais do Sistema, listados a seguir:

Tabela 2 – Requisitos não funcionais do sistema

Código	Descrição
RNF01	A interface gráfica deve refletir o reposicionamento e a reordenação de arquivos de forma imediata.
RNF02	O processamento das regras e aplicação de tetos deve possuir tempo de resposta ágil, sem travamentos na interface.
RNF03	O painel de gerenciamento administrativo deve ser restrito e protegido por controle de acesso (autenticação).

Fonte: Elaborado pelo autor (2026).

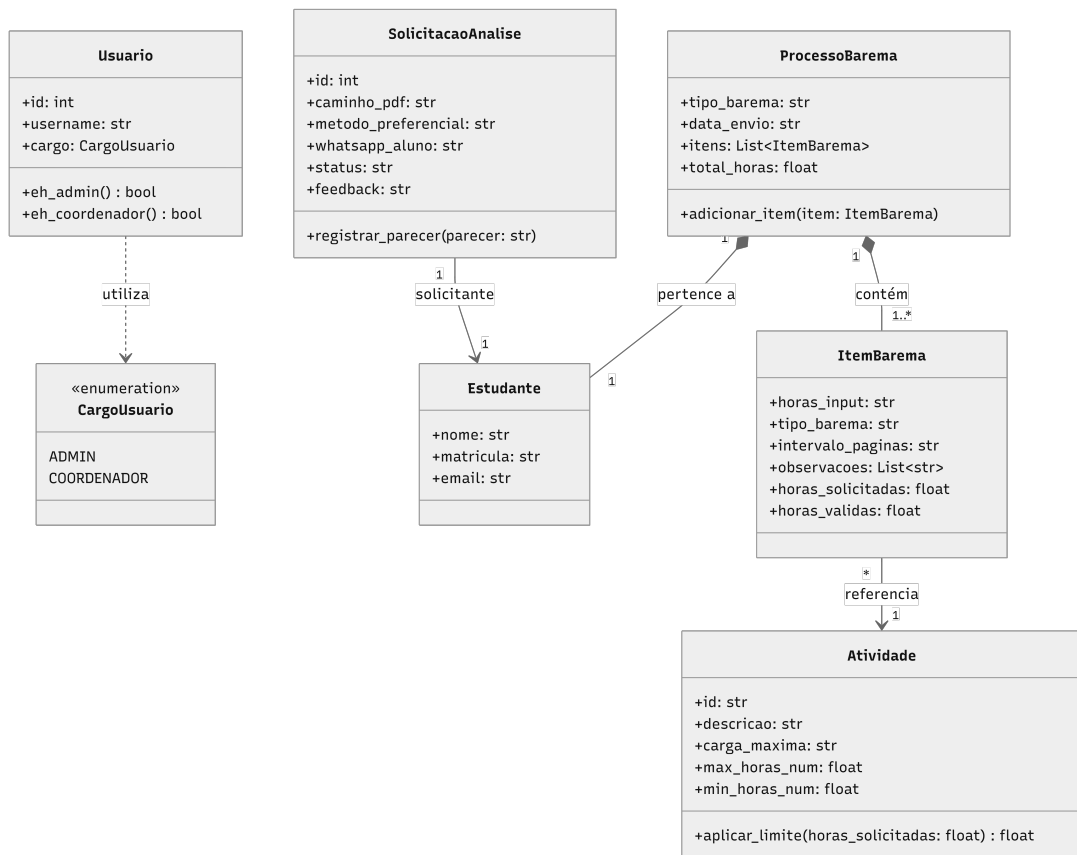
7.3 Arquitetura e Persistência de Dados no Projeto

A aplicação foi estruturada seguindo o padrão de arquitetura em camadas para isolar estritamente as responsabilidades de interação, controle e persistência. No *backend*, a orquestração das rotas e serviços foi implementada utilizando a linguagem PythonTM em conjunto com o *microframework* FlaskTM. A escolha por um *microframework* justifica-se pela necessidade do projeto em gerenciar estados voláteis na memória de forma ágil durante o rascunho do barema, sem a sobrecarga de ferramentas monolíticas.

Para refletir essa separação de responsabilidades no código-fonte, a arquitetura foi inspirada no padrão *Model-View-Controller* (MVC) aliado a princípios de isolamento de domínio. A base de código foi dividida em diretórios lógicos estruturais: o núcleo de regras de negócio foi isolado em `core/entities` (representando as estruturas de dados puras, desacopladas do banco) e `core/services` (que orquestra a lógica de extração e validação do Barema). Já a persistência e a comunicação HTTP foram delegadas a camadas externas, como os *controllers* de rotas e o `models.py`. Esse desacoplamento garante que a lógica de aplicação dos limites de horas não dependa do *framework web* ou do banco de dados.

Para guiar a implementação desse núcleo de negócio e documentar as entidades centrais, elaborou-se o Diagrama de Classes (Figura 3). A modelagem descreve as classes puras do sistema (como a Atividade, o Item do Barema e o Histórico de Solicitações), mapeando seus atributos, métodos de obtenção de dados e as funções responsáveis pela aplicação dos tetos regulamentares.

Figura 3 – Diagrama de Classes do núcleo de domínio do sistema.

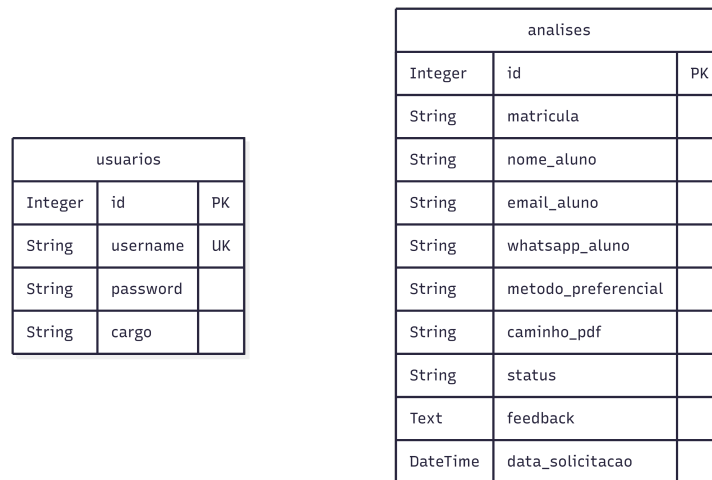


Fonte: Elaboração própria (2026).

O estado persistente do sistema — isto é, o armazenamento definitivo do histórico de solicitações e do vínculo com os arquivos gerados — foi materializado fisicamente através do banco de dados relacional *SQLite*. A comunicação entre os objetos de domínio em Python e as tabelas do banco de dados é mediada pelo *framework Flask-SQLAlchemy*. A adoção deste mapeador objeto-relacional (ORM) permitiu criar métodos de conversão bidirecional (padrão *Data Mapper*), de modo que as entidades fossem salvas no banco de dados abstraindo a complexidade de consultas SQL literais e assegurando transações atômicas seguras no registro dos protocolos submetidos.

O reflexo físico dessa persistência pode ser visualizado no Modelo Entidade-Relacionamento (MER) do banco de dados (Figura 4), que ilustra as tabelas geradas de forma automatizada pelo *SQLAlchemy* para suportar as análises dos discentes e as credenciais dos usuários administrativos.

Figura 4 – Modelo Entidade-Relacionamento (MER) do banco de dados.



Fonte: Elaboração própria (2026).

Por fim, ao observar o modelo estrutural do banco de dados, cabe destacar uma decisão importante sobre a arquitetura do projeto: o sistema não exige *login* dos discentes. Essa escolha foi feita para facilitar o uso, permitindo que o aluno gere seu protocolo de forma direta e sem burocracia. Por conta disso, as informações de identificação (nome, matrícula e *e-mail*) não ficam em uma tabela separada de usuários. Em vez disso, são salvas diretamente no registro da tabela de **analises**. Essa estratégia simplifica a infraestrutura, pois elimina a necessidade de gerenciar o cadastro e a recuperação de senhas dos alunos, restringindo a autenticação de segurança estritamente ao painel da coordenação.

7.4 Implementação

A partir do escopo definido pelos Casos de Uso, a construção do *software* foi segmentada em módulos operacionais. A seguir, detalha-se o funcionamento e as bibliotecas empregadas em cada um destes núcleos.

7.4.1 Montagem do Barema

Este módulo é responsável pela interação primária com o discente, englobando a coleta de dados e a recepção dos arquivos comprobatórios. Inicialmente, o usuário acessa uma tela com um formulário para inserção dos dados básicos de identificação. A matrícula inserida atua como chave de busca para que a lógica de domínio carregue dinamicamente o edital regulamentar adequado (Figura 5).

Figura 5 – Tela inicial do sistema Barema.

Barema de Atividades

Digite sua matrícula para carregar o formulário compatível com seu regulamento.

Monte seu Barema

Acesso administrativo

Nome do Discente:

Matrícula:

Email:

Data de Verificação:

Fonte: Elaboração própria (2026).

Após o preenchimento, a interface renderiza a tabela de atividades. A construção visual e interativa deste módulo foi realizada utilizando *JavaScript* nativo integrado à *Fetch API*. Essa tecnologia foi fundamental para atender ao requisito de usabilidade (RNF01), pois permite a comunicação assíncrona com o *backend*. Dessa forma, o aluno pode anexar PDFs e visualizar o preenchimento da carga horária sem que ocorra o recarregamento da página (Figura 6).

Figura 6 – Formulário do barema com atividades carregadas.

ATIVIDADE	C.H. MÁXIMA	C.H. CUMPRIDA	ANEXAR COMPROVANTE(S)
1. Participação no CACIC: coordenador (a) geral; membro de comissão de assuntos acadêmicos; membro de comissão de eventos; secretário (a); tesoureiro (a)	Máximo de 50 horas	0	Escolher Arquivo + Adicionar outro
2. Manutenção não remunerada dos laboratórios da computação	Máximo de 50 horas	0	Escolher Arquivo + Adicionar outro
3. Participação em eventos científicos relacionados à computação	Máximo de 80h desde que não seja inferior a 1 dia	0	Escolher Arquivo + Adicionar outro
4. Orientando em projeto de Iniciação Científica	Máximo de 60h/projeto	0	Escolher Arquivo + Adicionar outro
5. Atividades especiais apreciadas e aprovadas pelo COLCIC. Inclui, não exclusivamente, cursos de curta duração, presencial ou on-line, com no mínimo 12h desde que comprovado com certificado que possa ter a sua autenticidade verificada.	Máximo de 50h/ano/atividade	0	Escolher Arquivo + Adicionar outro

Fonte: Elaboração própria (2026).

7.4.2 Extração de Dados

Uma vez que o arquivo é recebido pela camada de rotas, o Módulo de Extração é acionado. Este componente foi otimizado para operar diretamente sobre o fluxo de *bytes* (*stream*) na memória volátil do servidor, dispensando a necessidade de gravação prévia em disco. Para garantir a resiliência da aplicação, o motor de extração foi arquitetado para lidar com dois cenários distintos de arquivos comprobatórios: documentos com camada de texto estruturada e imagens rasterizadas.

Para a inspeção de binários e extração nativa de metadados textuais em arquivos PDF padrão, o projeto empregou a biblioteca *PyMuPDF* (através do módulo `fitz`). O serviço instancia o documento na memória utilizando `fitz.open()` e percorre as páginas invocando métodos de captura direta de *strings* legíveis, o que garante máxima velocidade de processamento.

Contudo, para cenários em que o discente anexa arquivos de imagem (extensões como PNG ou JPEG) ou documentos PDF originados de *scanners* (sem camada de texto extraível), o sistema aciona um mecanismo de contingência baseado em Reconhecimento Óptico de Caracteres (OCR). Utilizando o motor *Tesseract OCR* (integrado via biblioteca `pytesseract`), o fluxo visual é processado e os caracteres contidos na imagem são reconhecidos e convertidos em texto estruturado na memória da aplicação.

Independentemente da via de extração acionada (nativa via *PyMuPDF* ou óptica via *OCR*), a *string* resultante é padronizada e submetida ao módulo nativo de Expressões Regulares do Python (`re`). O algoritmo utiliza a função `re.findall()` configurada com padrões morfológicos para localizar a carga horária declarada no texto, isolando números que estejam semanticamente adjacentes a termos específicos (como "horas" ou "h").

7.4.3 Pré-validação e Geração do Documento

Com os dados extraídos em memória, a aplicação executa a Pré-validação. A validação básica cruza os metadados do arquivo com as regras institucionais carregadas no edital, verificando discrepâncias na carga horária, titularidade e submissões em duplicidade (comparando o conteúdo bruto dos binários).

Além das verificações determinísticas, este módulo foi arquitetado para suportar uma camada opcional de contingência e flexibilidade textuais operada por recursos de inteligência artificial. Se a *API* externa estiver configurada e disponível, o sistema aproxima semanticamente o contexto do certificado com a categoria exigida; em caso de falha de conexão, o sistema adota um comportamento de resiliência e retorna à validação básica.

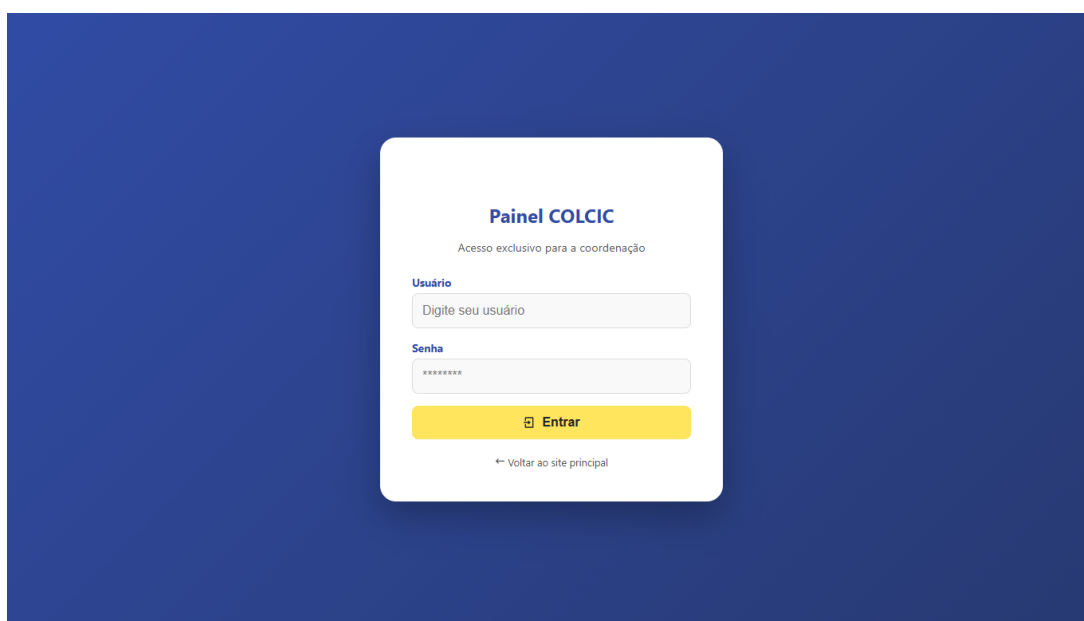
Superadas as validações, o processo culmina na geração do documento. Utilizando novamente os recursos de manipulação do *PyMuPDF*, o *backend* consolida os dados do formulário, cria laudas de indexação separadoras dinamicamente e anexa os comprovantes

validados, exportando um único PDF unificado e devidamente paginado para o aluno.

7.4.4 Painel de Análise (Coordenação)

O último componente do escopo lida com a tramitação oficial do documento gerado. Este módulo possui acesso restrito e requer autenticação, separando o fluxo público discente do acesso gerencial da coordenação (Figura 7).

Figura 7 – Tela de *login* administrativo.



Fonte: Elaboração própria (2026).

Ao confirmar o envio do barema, o Módulo de Análise converte os dados da memória transiente para o estado persistente, criando um registro de protocolo via SQLAlchemy na base SQLite. O coordenador, devidamente autenticado, acessa uma interface gerencial que consome esses registros, permitindo a filtragem de *status* e o acesso direto ao PDF unificado gerado no módulo anterior (Figura 8).

Figura 8 – Painel do coordenador para análise das solicitações.



Fonte: Elaboração própria (2026).

O ciclo de caso de uso deste módulo encerra-se com o registro do parecer institucional no sistema e a consequente notificação do discente. Para garantir a transparência do processo, o sistema implementa um serviço de mensageria acionado por gatilhos de evento (*event triggers*). Assim que o coordenador submete a avaliação e o *status* do protocolo é atualizado no banco de dados (acionando a função `registrar_parecer()`), a aplicação intercepta essa mudança de estado e invoca o módulo de comunicação.

Embasado nos dados persistidos, o sistema verifica o `metodo_preferencial` escolhido pelo discente durante o preenchimento. Para a orquestração e o disparo automático dessas mensagens, optou-se pela integração com a plataforma de comunicações em nuvem *Twilio*. A escolha dessa ferramenta justifica-se pela sua capacidade de centralizar múltiplos canais de notificação (*omnichannel*) em uma única API segura, escalável e de alta disponibilidade.

Caso a escolha do aluno seja por correio eletrônico, o *backend* aciona a API de *email* da plataforma (*Twilio SendGrid*) para despachar o *feedback* detalhado. Se a preferência for pelo aplicativo *WhatsApp*, o orquestrador consome a API de mensageria nativa do *Twilio*, entregando a notificação de forma síncrona e formatada diretamente no dispositivo móvel do discente - também é enviado o feedback por *email* -. Essa automatização conclui o fluxo do projeto, eliminando por completo a carga de trabalho operacional da coordenação com o envio manual de retornos e garantindo que o discente seja informado de imediato sobre o encerramento institucional do seu processo.

8 Resultados e Discussão

Este capítulo apresenta os resultados obtidos com a implementação do protótipo, os cenários detalhados utilizados para sua homologação e uma discussão técnica sobre o papel da solução na preparação automatizada do Barema do COLCIC. A estratégia de validação adotada neste trabalho segue o planejamento previsto e os parâmetros definidos na Seção 6.3.1, confrontando as saídas geradas pelo sistema com os critérios de sucesso estabelecidos para a aplicação. Todos os resultados descritos concentram-se em evidências funcionais mensuradas em ambiente de testes local.

8.1 Resultados Obtidos

O principal resultado prático deste trabalho consiste na consolidação de uma plataforma *web* funcional projetada especificamente para mediar a preparação de documentos acadêmicos regulatórios. O sistema foi estruturado como um ambiente transiente de trabalho, permitindo que o discente consolide seus dados de matrícula, selecione o barema de transição aplicável, informe as horas declaradas e anexe os comprovantes correspondentes de forma assistida.

Para gerenciar as interações com os arquivos sem sobrecarregar o servidor, o sistema utiliza uma *interface* baseada em componentes reativos em *JavaScript* que se comunicam de maneira assíncrona com a camada de serviços do FlaskTM. O usuário realiza a anexação de novos certificados através de campos de *upload* dinâmicos; o sistema intercepta o arquivo e o envia via *Fetch API* para o *backend*, onde o orquestrador o aloca em uma lista de objetos temporária na memória da sessão. Para substituir ou remover documentos, o usuário conta com ícone e botões intuitivos na *interface*. A substituição e a remoção de documentos ocorrem por meio da manipulação direta dos índices dessa lista volátil, garantindo respostas visuais imediatas na interface sem interrupções ou necessidade de recarregamento total da página.

Outro resultado relevante foi a estruturação do mecanismo de inspeção técnica de binários operado pela biblioteca PyMuPDFTM. No instante em que um certificado é posicionado em uma categoria, o servidor abre o fluxo de *bytes* do arquivo diretamente na memória e executa rotinas baseadas em expressões regulares para identificar padrões textuais de carga horária e datas de realização. Esse processamento imediato alimenta o motor de regras da aplicação, permitindo que o sistema atualize os contadores visuais e aplique os tetos regulamentares na *interface* de maneira instantânea.

A integração entre a extração e a pré-validação de dados demonstrou utilidade

prática ao atuar como um filtro de qualidade antecedente à submissão oficial. Ao cruzar as informações textuais do PDF com as declarações do estudante, a plataforma identifica e bloqueia inconsistências operacionais, tais como divergências ortográficas entre o nome do aluno e o emitido no certificado, tentativas de declarar cargas horárias superiores ao total textual comprovado no documento e anexação de arquivos duplicados, cuja similaridade é atestada pela comparação do conteúdo bruto extraído de cada binário.

8.2 Validação do Protótipo

A validação do protótipo foi conduzida por meio de testes de conformidade normativa, simulando cenários reais enfrentados pela coordenação do curso. A estrita aderência às regras do Barema é mandatória, visto que o documento final possui valor regulatório e subsidia o deferimento da colação de grau pelo Colegiado; qualquer erro de cálculo ou categorização inadequada resulta no indeferimento do processo e em prejuízos ao cronograma acadêmico do discente. Para cobrir essas variáveis, foram projetados cenários de testes funcionais, explicados a seguir.

Nos cenários avaliados, o sistema conseguiu identificar inconsistências objetivas antes da submissão oficial. A divergência de nome foi detectada pela comparação entre o nome informado pelo discente e o nome identificado no certificado. A carga horária solicitada foi comparada com o total extraído dos documentos anexados, permitindo apontar quando o valor declarado era superior ao comprovado. A duplicidade foi verificada pela comparação do conteúdo textual dos certificados.

Também foi testado o comportamento da pré-validação quando há indisponibilidade de serviços externos de inteligência artificial. Nesse caso, o sistema retornou para a validação básica, mantendo a análise de regras objetivas sem depender exclusivamente de uma *API* externa. Esse comportamento é relevante porque preserva a utilidade do sistema mesmo quando recursos adicionais de análise textual não estão disponíveis.

Além das validações individuais, foi verificada a geração do documento final. O PDF consolidado reuniu os dados preenchidos e os certificados anexados em uma estrutura única, adequada para encaminhamento à análise oficial. Esse resultado atende à necessidade de reduzir erros de montagem e de organização dos anexos.

8.3 Discussão da Solução

Os resultados indicam que a solução proposta se diferencia de ferramentas genéricas de manipulação de PDF por incorporar regras específicas do barema do COLCIC. Enquanto ferramentas externas apenas permitem juntar ou reorganizar arquivos, o sis-

tema desenvolvido combina a montagem do documento com verificações relacionadas ao regulamento e ao conteúdo dos certificados.

A arquitetura em camadas adotada conferiu flexibilidade e isolamento de escopo ao projeto. A separação clara entre a camada de *interface*, as rotas do Flask™, as funções de serviço do orquestrador e os *drivers* de geração de documentos permitiu que os limites normativos fossem codificados diretamente no *backend* através de um dicionário de restrições explícito. Essa organização facilitou a escrita de rotinas de testes unitários automatizados, validando isoladamente os módulos como cálculo dos pisos e tetos das atividades, extração de carga horária e pré-validação.

Apesar dos resultados promissores, a plataforma deve ser interpretada como um instrumento de apoio à decisão e conferência preliminar, não substituindo o papel avaliador da coordenação do curso. O sistema reduz os erros formais e inconsistências matemáticas evidentes antes do envio, mas a prerrogativa pedagógica de deferimento das atividades e análise de mérito dos certificados permanece estritamente institucional.

8.4 Limitações Observadas

Antes de delimitar as restrições do cenário atual, cabe destacar a robustez técnica demonstrada pelo protótipo no processamento estável de fluxos assíncronos e na fidelidade do documento consolidado gerado. Contudo, foram mapeadas limitações operacionais inerentes à tecnologia empregada que devem ser documentadas.

A primeira limitação reside na dependência da qualidade estrutural do arquivo PDF enviado. A extração automática de dados textuais via PyMuPDF™ pressupõe que o documento possua uma camada de texto digitalizada; certificados que consistem em imagens puras ou escaneamentos de baixa resolução demandam o uso de ferramentas adicionais de Reconhecimento Óptico de Caracteres (OCR), funcionalidade que não integra o escopo da versão atual. Ademais, as rotinas que utilizam chamadas de inteligência artificial externa ficam condicionadas à latência de rede e à estabilidade do provedor escolhido, reforçando a importância do mecanismo de contingência estruturado no *backend*.

Por fim, destaca-se que todas as homologações foram realizadas em ambiente local controlado. Embora os cenários simulados reflitam com precisão as regras do COLCIC, a ausência de testes de usabilidade com uma amostra estatisticamente heterogênea de discentes e coordenadores impede a coleta de métricas empíricas sobre a redução real de tempo administrativo e a curva de aprendizado da *interface* em produção, configurando uma oportunidade para trabalhos futuros.

9 Conclusão

Este trabalho apresentou a concepção e a implementação de um protótipo de sistema web para apoiar a preparação do Barema de Atividades Complementares do curso de Ciência da Computação da UESC. O sistema proposto busca reduzir a burocracia do processo manual, aplicando automaticamente as regras de negócio do regulamento do COLCIC e consolidando os comprovantes em um único PDF paginado.

A partir da análise do problema, ficou evidente que o processo atual impõe três desafios principais: cálculo manual de horas, indexação de páginas sensível a alterações na ordem dos documentos e retrabalho administrativo causado por erros frequentes. A solução desenvolvida atua diretamente sobre esses pontos, oferecendo:

- extração automática de metadados de arquivos PDF;
- aplicação de limites de horas por categoria;
- geração de um documento final unificado com indexação e paginação automática;
- pré-validação de inconsistências antes da submissão oficial.

Os resultados indicam que o protótipo tem potencial para melhorar a confiabilidade do Barema e reduzir o trabalho repetitivo do aluno e da coordenação. A escolha de um modelo arquitetural em camadas, com separação de responsabilidades entre interface, orquestração de regras de negócio e persistência, favoreceu a manutenção do código e a escalabilidade da solução.

Por outro lado, foram identificadas limitações que devem ser consideradas em versões futuras:

- necessidade de validações adicionais de usabilidade junto a discentes e coordenadores;
- inexistência, na versão atual, de uma análise formal de desempenho em ambiente de produção;
- falta de integração completa com o fluxo oficial de envio acadêmico do colegiado.

Como trabalho futuro, sugere-se ampliar a solução com testes de usabilidade reais junto a discentes e coordenação, integrar o fluxo com sistemas institucionais de autenticação e protocolo acadêmico, e evoluir o painel administrativo com relatórios de análise e monitoramento de desempenho, de modo a transformar o protótipo em uma plataforma mais robusta e aderente ao processo oficial do COLCIC.

Em síntese, a plataforma desenvolvida demonstra que é viável automatizar uma etapa relevante do processo de reconhecimento de Atividades Complementares, entregando uma solução com valor prático para alunos e coordenação, ao mesmo tempo em que preserva a condição de apoio à análise oficial do colegiado.

Referências

ABIDEMI, A. The role of technology and automation in streamlining business processes and productivity for SMEs. *International Journal of Entrepreneurship*, v. 7, n. 3, p. 31, 2024. ISSN 2510-4100. Acesso em: mar. 2026. Disponível em: <https://ajpojournals.org/journals/ije/article/view/2510/4100>. 22

HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. *Introduction to Automata Theory, Languages, and Computation*. 3. ed. Boston: Addison-Wesley, 2006. ISBN 978-0-321-46225-1. 25

iLovePDF. *Ferramentas de PDF online para amantes de PDF*. 2026. Acessado em: 12 abr. 2026. Disponível em: <https://www.ilovepdf.com/pt>. 17

LAUDON, K. C.; LAUDON, J. P. *Sistemas de Informação Gerenciais*. 11. ed. [S.l.]: Pearson Education, 2014. 17

NIELSEN, J. *Response Times: The 3 Important Limits*. Nielsen Norman Group, 1993. Acessado em: 04 abr. 2026. Disponível em: <https://www.nngroup.com/articles/response-times-3-important-limits/>. 19, 25

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: Uma Abordagem Profissional*. 8. ed. Porto Alegre: AMGH, 2016. Disponível em: Internet Archive. Acessado em: 12 mar. 2026. Disponível em: <https://archive.org/details/pressman-engenharia-de-software-uma-abordagem-profissional-8a>. 22, 24

SOARES, B. C.; RESENDE, R. S. F. *Requisitos para utilização de prototipagem evolutiva nos processos de desenvolvimento de software para Web*. Belo Horizonte, MG, 2007. 27

TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. 4. ed. São Paulo: Pearson Education do Brasil, 2016. Acessado em: 13 mar. 2026. Disponível em: <https://archive.org/details/SistemasOperacionaisModernosTanenbaum4Edio/>. 18